Stler Sudoku

Technical Documentation

Version: 1.3 (versionCode 5) · April 2026

Platform: Android · Min SDK 26 (Android 8.0)

Repository: github.com/JuliaSivridi/Sudoku

Stack: Kotlin · Jetpack Compose · Material 3 · ViewModel + StateFlow

Summary: A clean, relaxing Sudoku app for Android. Fully offline, no ads, no accounts. Features three difficulty levels, two input modes, four color themes, notes with auto-clue filling, conflict highlighting, an in-app solver, and automatic game save/restore.

Table of Contents

1. Overview
2. Tech Stack
3. Project Architecture
4. Data Model
5. State Management
6. Game Logic — Generator & Solver
7. Screens
8. UI Components
9. Theme & Colors
10. Persistence
11. Settings & Preferences

1. Overview

Stler Sudoku is a personal Android application built entirely with **Kotlin and Jetpack Compose**. It runs fully offline — no network calls, no accounts, no analytics. All state is persisted locally via **SharedPreferences**.

Key Design Goals

- **Relaxing UX** — no timers, no penalties, no pressure
- **Two input modes** — Number first (digit → cell) and Cell first (cell → digit), switchable at any time
- **Full undo history** — every move is reversible back to the start of the game
- **Adaptive theming** — four accent palettes, dark/light mode follows system setting
- **Unique puzzles** — generator guarantees exactly one solution per puzzle

Feature Summary

Feature	Description
Difficulty levels	Easy (46 givens), Medium (36), Hard (29)
Input modes	Number first / Cell first — saved to SharedPreferences
Notes	3×3 mini-grid per cell; auto-cleared on digit placement in same row/col/box
Clues toggle	Fills all valid candidates across the board; toggles off to clear all notes
Conflict highlight	Red text + tinted background for duplicate digits (user-placed only)
Undo	Full history to game start; unlimited depth (capped at 50 states)
Hints	Reveals a random correct digit; unlimited
Save & Continue	Auto-saves after every move via StateFlow collector
Statistics	Counts completed games per difficulty
Solver	Standalone screen — enter any puzzle and tap Solve
Themes	Orange / Green / Blue / Purple; persisted to SharedPreferences

2. Tech Stack

Layer	Technology	Notes
Language	Kotlin	Coroutines, data classes, extension functions
UI framework	Jetpack Compose	Material 3; no XML layouts
Architecture	MVVM — ViewModel + StateFlow	`AndroidViewModel`; `collectAsState()` in composables
Navigation	Navigation Compose	Typed routes with query parameters
Theming	CompositionLocal	`LocalAppThemeColors` propagates accent palette
Persistence	SharedPreferences + org.json	Game save, stats, theme, input preference
Icons	Material Icons Extended	Outlined + Filled variants
Min SDK	API 26 (Android 8.0)	Target SDK 36

3. Project Architecture

```
com.example.sudoku/
├── MainActivity.kt # entry point; owns ThemeViewModel + InputPreferenceViewModel
├── data/ # SharedPreferences managers
│ ├── GameSaveManager.kt # serialize/deserialize GameState to JSON
│ ├── InputPreferenceManager.kt# persists NUMBER_FIRST / CELL_FIRST
│ ├── StatsManager.kt # completed game counts per Difficulty
│ └── ThemeManager.kt # persists AppColorTheme enum
├── game/ # pure logic — no Android dependencies
│ ├── SudokuGenerator.kt # generates puzzle + solution pair
│ └── SudokuSolver.kt # backtracking solver; isValid, solve, countSolutions
├── model/ # data classes and enums
│ ├── Cell.kt # value, isGiven, notes: Set<Int>
│ ├── Difficulty.kt # EASY / MEDIUM / HARD + givens count
│ ├── GameState.kt # full immutable game snapshot
│ ├── InputMode.kt # NORMAL / NOTES / ERASE
│ └── InputPreference.kt # NUMBER_FIRST / CELL_FIRST
├── navigation/
│ └── AppNavigation.kt # NavHost; all routes; collects ViewModels
├── ui/
│ ├── components/ # reusable composables used inside GameScreen
│ │ ├── ControlButtons.kt # Undo / Clear / Notes / Clues / Hint row
│ │ ├── NumberRow.kt # 3×3 digit pad
│ │ └── SudokuGrid.kt # 9×9 board + NotesGrid + hasConflict
│ ├── screens/
│ │ ├── StartScreen.kt # difficulty picker + bottom nav
│ │ ├── GameScreen.kt # main gameplay screen
│ │ ├── EndScreen.kt # motivational finish screen
│ │ ├── SolverScreen.kt # standalone puzzle solver
│ │ ├── StatisticsScreen.kt # games played per difficulty
│ │ └── SettingsScreen.kt # theme + input mode picker
│ └── theme/
│ ├── Color.kt # all color constants + 4 AppThemeColors instances
│ ├── Theme.kt # SudokuTheme composable; builds ColorScheme dynamically
│ └── Type.kt # typography
└── viewmodel/
├── GameViewModel.kt # all gameplay mutations; auto-save collector
├── InputPreferenceViewModel.kt # StateFlow<InputPreference>
├── SolverViewModel.kt # solver screen state
└── ThemeViewModel.kt # StateFlow<AppColorTheme>
```

Data Flow

1. **App start** → `MainActivity` creates `ThemeViewModel` and `InputPreferenceViewModel`, reads persisted values, wraps content in `SudokuTheme`
2. **Navigation** → `AppNavigation` owns `GameViewModel` and `SolverViewModel`; passes `inputPreference` to `GameScreen`
3. **Game mutation** → composable calls ViewModel method → ViewModel updates `_state: MutableStateFlow<GameState>`
4. **Recomposition** → `collectAsState()` in screen triggers UI update
5. **Auto-save** → `init` block launches `state.collect { … }`; saves to SharedPreferences after every state change

4. Data Model

Cell

```
data class Cell(
    val value: Int = 0,            // 0 = empty
    val isGiven: Boolean = false,  // true = pre-filled; user cannot modify
    val notes: Set<Int> = emptySet() // pencil marks 1–9
)
```

Difficulty

```
enum class Difficulty(val label: String, val givens: Int) {
    EASY("Easy",   46),   // 35 empty cells
    MEDIUM("Medium", 36), // 45 empty cells
    HARD("Hard",   29)    // 52 empty cells
}
```

InputMode

```
enum class InputMode { NORMAL, NOTES, ERASE }
```

`ERASE` is only used in **Number first** mode. In **Cell first** mode the Clear button erases the selected cell directly without entering a mode.

InputPreference

```
enum class InputPreference { NUMBER_FIRST, CELL_FIRST }
```

GameState

Field	Type	Notes
board	`List<List<Cell>>`	9×9 immutable grid; replaced on every mutation
solution	`List<List<Int>>`	Full correct solution; used for hint and completion check
difficulty	`Difficulty`	

selectedCell	`Pair<Int,Int>?`	Row, col of highlighted cell
selectedDigit	`Int?`	Active digit for matching highlight and placement
inputMode	`InputMode`	Current input mode; ERASE unused in Cell first
isComplete	`Boolean`	Set when all cells match solution; triggers EndScreen
autoNotesActive	`Boolean`	Clues toggle state; independent of undo stack
undoStack	`List<List<List<Cell>>>`	Board snapshots; max depth 50; most recent at end

5. State Management

All mutable state lives in `AndroidViewModel` subclasses exposed as `StateFlow`. Composables collect state with `collectAsState()`.

ViewModel	StateFlow	Responsibility
`GameViewModel`	`GameState`	All gameplay: cell tap, digit select, undo, hint, notes, clues, auto-save, stats
`ThemeViewModel`	`AppColorTheme`	Accent palette; persisted via `ThemeManager`
`InputPreferenceViewModel`	`InputPreference`	Input mode (Number first / Cell first); persisted via `InputPreferenceManager`
`SolverViewModel`	`GameState`	Solver screen — independent board, no save/stats

Auto-save Pattern

The save happens inside a `StateFlow` collector launched in `GameViewModel.init`, ensuring it fires synchronously with every state emission — even if Android kills the process shortly after:

```
init {
    viewModelScope.launch {
        state.collect { gameState ->
            when {
                gameState.isComplete -> {
                    saveManager.clearSavedGame()
                    if (!statsRecorded) {
                        statsRecorded = true
                        statsManager.increment(gameState.difficulty)
                    }
                }
                gameState.board.any { row -> row.any { !it.isGiven && it.value != 0 } } -
                    saveManager.saveGame(gameState)
            }
        }
    }
}
```

Input Mode Routing (GameScreen)

Based on `InputPreference`, the UI routes to different ViewModel methods:

Action	Number first	Cell first
Tap cell	`onCellTap(row, col)` — places digit if selected	`selectCell(row, col)` — only selects
Tap digit pad	`selectDigit(digit)` — sets selectedDigit, clears selectedCell	`placeDigit(digit)` — places in selectedCell immediately
Tap Clear	`toggleErase()` — toggles ERASE mode	`eraseSelected()` — direct action, no mode change

> **Digit highlight sync:** Tapping a filled cell sets `selectedDigit` to that cell's value in both modes, highlighting all matching digits on the board.

6. Game Logic — Generator & Solver

SudokuSolver

A pure Kotlin object with three functions used by both generation and the Solver screen:

Function	Description
`isValid(board, row, col, num)`	Checks row, column and 3×3 box for conflicts
`solve(board)`	In-place backtracking solver; returns `true` if solved
`countSolutions(board, limit=2)`	Counts solutions up to `limit`; stops early — used to verify uniqueness

SudokuGenerator — Puzzle Generation

Two-phase algorithm:

Phase 1 — Full board generation

Recursive backtracking with **randomised digit order** at each cell. For each empty cell, the digits 1–9 are shuffled before trying, producing a different valid complete board on every call.

Phase 2 — Cell removal

Cells are removed in random order. After each removal, `SudokuSolver.countSolutions()` is called with `limit = 2`. If the count is still 1 (unique solution), the cell stays empty. Otherwise it is restored. Removal continues until the target number of empty cells is reached:

```
val cellsToRemove = 81 - difficulty.givens
// EASY:   81 - 46 = 35 cells removed
// MEDIUM: 81 - 36 = 45 cells removed
// HARD:   81 - 29 = 52 cells removed
```

> **Uniqueness guarantee:** Every generated puzzle has exactly one solution. The `countSolutions` early-exit at 2 keeps the check fast — worst case is still exponential but terminates as soon as a second solution branch is found.

Conflict Detection (hasConflict)

Implemented in `SudokuGrid.kt` as a private function. A cell is highlighted red when:

- It is **not** a given cell (`isGiven == false`)

- Its value matches any other cell in the same row, column, or 3×3 box (given or user-placed)

Given cells are never highlighted — the early `if (cell.isGiven) return false` prevents it.

Auto-notes (Clues)

When Clues is toggled on, every empty cell's valid candidates are computed on the fly:

```
val possible = (1..9).filter { digit ->
    canPlace(current.board, row, col, digit)
}.toSet()
cell.copy(notes = possible)
```

`canPlace` checks row, column and box — identical logic to `SudokuSolver.isValid`. Toggling off maps all cells to `cell.copy(notes = emptySet())`. Both operations are pushed to the undo stack. The `autoNotesActive` flag is intentionally *not* reset by undo.

7. Screens

StartScreen

Layout (vertical, centered):

- Title **SUDOKU** in accent color
- **Continue** button (visible only when a saved game exists — checked via `GameSaveManager.hasSavedGame()`)
- 56 dp spacer → "Select difficulty" subtitle → Easy / Medium / Hard buttons
- 56 dp spacer → **Solver** button
- Bottom navigation bar: Home / Stats / Settings

GameScreen

The main gameplay screen. No Scaffold — plain `Column` on background color.

- Back button + difficulty label header
- `SudokuGrid` — full-width, `aspectRatio(1f)`
- `ControlButtons` — Undo / Clear / Notes / Clues / Hint
- `NumberRow` — 3×3 digit pad

EndScreen

Shown on game completion. Displays a random motivational message. Buttons: **Play Again** (same difficulty) and **Menu**.

SolverScreen

Independent instance of the grid backed by `SolverViewModel`. All cells are user-editable (no given cells). Tap **Solve** to run `SudokuSolver.solve()`. All placed digits render in accent color.

StatisticsScreen

Reads counts from `StatsManager`. Displays a bordered table: Mode | Completed. Title: "Games Played".

SettingsScreen

- **Input mode** — `SingleChoiceSegmentedButtonRow`: "Number first" / "Cell first"
- **Theme** — four full-width buttons, each in its own accent color; active theme shows a checkmark

8. UI Components

SudokuGrid

A `Box` with `aspectRatio(1f)`. Cell backgrounds are composable layers; grid lines are drawn on top via a `drawWithContent` Canvas overlay, ensuring borders are never obscured by cell fills.

Cell background priority (layered)

1. Row/column highlight (`CellHighlight*`) if cell is in the same row or column as `selectedCell`
2. Digit match highlight (`cellDigitHighlight*`, ~20–50% alpha accent) if cell value equals `selectedDigit`
3. Selected cell (`CellSelected*`) if this is `selectedCell`
4. Conflict background (`ConflictBg*`, ~20–50% alpha red) layered on top of any of the above

Grid border rendering

Border type	Stroke	Color
Outer frame	2 dp	`GridOuterBorder*`
3×3 box boundaries (i % 3 == 0)	2 dp	`GridOuterBorder*`
Inner cell lines	1 dp	`GridInnerBorder*`

Notes mini-grid

When a cell has notes and no value, a fixed 3×3 `Column/Row` is rendered. Digit N occupies position `row = (N-1)/3`, `col = (N-1)%3` — so 1 is always top-left and 9 always bottom-right, regardless of which notes are present.

NumberRow

A 3×3 grid of digit cells. Each cell:

- Width = 1/3 of total width (matching a 3×3 box in the grid above)
- `aspectRatio(2f)` — height is half the width (~1.5× the old single-row height)
- Font size: 32 sp
- Disappears when all 9 of that digit are placed (`digitCounts[digit] >= 9`)
- Selection highlight suppressed in Cell first mode (`showSelection` parameter)

ControlButtons

Five equal-weight columns in a `Row`. Each button is a full-width `Column` (icon + label) — the clickable area covers the entire column width, avoiding missed taps.

Button	Icon	Behaviour
Undo	Undo	Restores last board snapshot from undo stack
Clear	Close (×)	Number first: toggles ERASE mode. Cell first: immediately clears selected cell
Notes	EditNote	Toggles NOTES input mode (both modes)
Clues	AutoAwesome	Toggle: ON fills all valid candidates; OFF clears all notes
Hint	Lightbulb	Reveals correct digit in a random empty cell

9. Theme & Colors

System

Dark / light mode follows the OS setting (`isSystemInDarkTheme()`). The root `Surface` in `MainActivity` explicitly uses `MaterialTheme.colorScheme.background` so all screens share the same background regardless of whether they use `Scaffold`.

Base colors (non-theme)

Constant	Light	Dark	Usage
Background	#FFFFFF	#1C1C1C	Screen background
Surface	#FFFFFF	#363636	Cards, navigation bar
OnBackground	#1C1C1C	#EEEEEE	Primary text
CellHighlight	#F0F0F0	#363636	Row / column highlight
CellSelected	#E4E4E4	#424242	Tapped cell background
GridOuterBorder	#4D5666	#656D7A	Box and outer borders
GridInnerBorder	#BEC5D1	#363C47	Cell separator lines
NoteNumber	#909090	#9E9E9E	Pencil mark digits
ConflictColor	#CC1515	#FF5252	Error digit text
ConflictBg	#33CC1515 (~20%)	#80CC1515 (~50%)	Error cell background tint

AppThemeColors — per-theme values

Theme	accent	accentVariant
Orange (default)	#E07E38	#E8935A
Green	#4D8B53	#79B57F
Blue	#3D73B0	#6B9FD4
Purple	#7A4FA3	#A47BC8

Each theme also provides `cellDigitHighlightLight` (33% alpha accent) and `cellDigitHighlightDark` (50% alpha accent) for the digit-match cell tint.

Theme propagation

`SudokuTheme` in `Theme.kt` builds a Material 3 `ColorScheme` dynamically from the current `AppThemeColors`, then wraps content in `CompositionLocalProvider(LocalAppThemeColors provides themeColors)`. Components access per-theme colors via `LocalAppThemeColors.current`.

10. Persistence

All persistence uses a single `SharedPreferences` file named `"sudoku_prefs"`.

Manager	Key	Format
`GameSaveManager`	`"saved_game"`	JSON string (see below)
`StatsManager`	`"stats_{DIFFICULTY}"`	Integer count per difficulty name
`ThemeManager`	`"color_theme"`	`AppColorTheme.name` string
`InputPreferenceManager`	`"input_preference"`	`InputPreference.name` string

GameSaveManager — JSON Schema

```
{
  "difficulty": "MEDIUM",
  "board": [                        // 9×9 array
    [ {"v": 5, "g": true,  "n": []},
      {"v": 0, "g": false, "n": [1, 3, 7]},
      ... ],
    ...
  ],
  "solution": [                     // 9×9 array of ints
    [5, 3, 4, 6, 7, 8, 9, 1, 2],
    ...
  ]
}
```

Cell fields: `v` = value (0 = empty), `g` = isGiven, `n` = notes array. The undo stack and UI state (selectedCell, selectedDigit, inputMode) are not saved — the game resumes in a clean NORMAL state.

Auto-save trigger

Saving happens inside a `StateFlow` collector in `GameViewModel.init` (not in `onCleared()`, which is unreliable when Android kills the process from the recents screen). The game is saved whenever at least one non-given cell has a value. On completion, the save is cleared and stats are incremented exactly once per game (guarded by the `statsRecorded` flag).

11. Settings & Preferences

Theme

`ThemeViewModel` holds a `MutableStateFlow<AppColorTheme>` initialised from `ThemeManager`. `MainActivity` collects it and passes the value to `SudokuTheme`, triggering a full recomposition — the accent color changes immediately without restarting the app.

Input Preference

`InputPreferenceViewModel` follows the same pattern. The value is collected in `AppNavigation` and passed as a parameter to `GameScreen`, where it routes cell-tap and digit-tap callbacks to different ViewModel methods:

Preference	Digit pad highlight	Notes multi-tap	Clear button
Number first	Shows selected digit	Must re-select cell each time	Toggles ERASE mode
Cell first	Hidden	Select cell once, tap digits freely	Direct erase action

> **Default:** Number first. New users familiar with classic Sudoku apps will find the default behaviour matches their expectation. Cell first is recommended for players who prefer filling multiple notes in one cell without switching focus.